

172

通常情况下，语句是顺序执行的。但除非是最简单的程序，否则仅有顺序执行远远不够。因此，C++语言提供了一组控制流（flow-of-control）语句以支持更复杂的执行路径。



## 5.1 简单语句

C++语言中的大多数语句都以分号结束，一个表达式，比如 `ival + 5`，末尾加上分号就变成了**表达式语句**（expression statement）。表达式语句的作用是执行表达式并丢弃掉求值结果：

```
ival + 5;           // 一条没什么实际用处的表达式语句
cout << ival;       // 一条有用的表达式语句
```

第一条语句没什么用处，因为虽然执行了加法，但是相加的结果没被使用。比较普遍的情况是，表达式语句中的表达式在求值时附带有其他效果，比如给变量赋了新值或者输出了结果。

### 空语句

最简单的语句是**空语句**（null statement），空语句中只含有一个单独的分号：

```
; // 空语句
```

如果在程序的某个地方，语法上需要一条语句但是逻辑上不需要，此时应该使用空语句。一种常见的情况是，当循环的全部工作在条件部分就可以完成时，我们通常会用到空语句。例如，我们想读取输入流的内容直到遇到一个特定的值为止，除此之外什么事情也不做：

```
// 重复读入数据直至到达文件末尾或某次输入的值等于 sought
while (cin >> s && s != sought)
    ; // 空语句
```

while 循环的条件部分首先从标准输入读取一个值并且隐式地检查 cin，判断读取是否成功。假定读取成功，条件的后半部分检查读进来的值是否等于 sought 的值。如果发现了想要的值，循环终止；否则，从 cin 中继续读取另一个值，再一次判断循环的条件。



使用空语句时应该加上注释，从而令读这段代码的人知道该语句是有意省略的。

### 别漏写分号，也别多写分号

因为空语句是一条语句，所以可用在任何允许使用语句的地方。由于这个原因，某些看起来非法的分号往往只不过是一条空语句而已，从语法上说得过去。下面的片段包含两条语句：表达式语句和空语句。

173

```
ival = v1 + v2;; // 正确：第二个分号表示一条多余的空语句
```

多余的空语句一般来说是无害的，但是如果在 if 或者 while 的条件后面跟了一个额外的分号就可能完全改变程序员的初衷。例如，下面的代码将无休止地循环下去：

```
// 出现了糟糕的情况：额外的分号，循环体是那条空语句
while (iter != svec.end()) ;           // while 循环体是那条空语句
    ++iter;                             // 递增运算不属于循环的一部分
```

虽然从形式上来看执行递增运算的语句前面有缩进，但它并不是循环的一部分。循环条件后面跟着的分号构成了一条空语句，它才是真正的循环体。



多余的空语句并非总是无害的。

### 复合语句（块）

**复合语句（compound statement）**是指用花括号括起来的（可能为空的）语句和声明的序列，复合语句也被称作**块（block）**。一个块就是一个作用域（参见 2.2.4 节，第 43 页），在块中引入的名字只能在块内部以及嵌套在块中的子块里访问。通常，名字在有限的区域内可见，该区域从名字定义处开始，到名字所在的（最内层）块的结尾为止。

如果在程序的某个地方，语法上需要一条语句，但是逻辑上需要多条语句，则应该使用复合语句。例如，while 或者 for 的循环体必须是一条语句，但是我们常常需要在循环体内做很多事情，此时就需要将多条语句用花括号括起来，从而把语句序列转变成块。

举个例子，回忆 1.4.1 节（第 10 页）的 while 循环：

```
while (val <= 10) {  
    sum += val;      // 把 sum + val 的值赋给 sum。  
    ++val;          // 给 val 加 1  
}
```

程序从逻辑上来说要执行两条语句，但是 while 循环只能容纳一条。此时，把要执行的语句用花括号括起来，就将其转换成了一条（复合）语句。

Note

块不以分号作为结束。

所谓空块，是指内部没有任何语句的一对花括号。空块的作用等价于空语句：

```
while (cin >> s && s != sought)  
{ } // 空块
```

## 5.1 节练习

174

**练习 5.1:** 什么是空语句？什么时候会用到空语句？

**练习 5.2:** 什么是块？什么时候会用到块？

**练习 5.3:** 使用逗号运算符（参见 4.10 节，第 140 页）重写 1.4.1 节（第 10 页）的 while 循环，使它不再需要块，观察改写之后的代码的可读性提高了还是降低了。

## 5.2 语句作用域

可以在 if、switch、while 和 for 语句的控制结构内定义变量。定义在控制结构当中的变量只在相应语句的内部可见，一旦语句结束，变量也就超出其作用范围了：

```
while (int i = get_num()) // 每次迭代时创建并初始化 i  
    cout << i << endl;  
i = 0; // 错误：在循环外部无法访问 i
```